Applied Machine Learning — Week 2

Linear Models & Regularisation · Kernel Methods · Optimisation

Comprehensive Revision Notes

Contents

Part I
Linear Models & Regularisation

Linear Regression

Model Setup

A linear model predicts $\hat{y}_i = \boldsymbol{w}^\top \phi(x_i)$, where $\phi : \mathbb{R}^d \to \mathbb{R}^D$ is a **feature map**. The model is linear *in the weights* $\boldsymbol{w}$, not necessarily in the raw inputs x (e.g. polynomial features).

Key Notation

- Data: $\{(x_i, y_i)\}_{i=1}^N$, inputs $x_i \in \mathbb{R}^d$, targets $y_i \in \mathbb{R}$.
- Feature map: $\phi(x_i) \in \mathbb{R}^D$; includes a constant 1 for the intercept.
- **Design matrix**: $\Phi = \begin{pmatrix} -\phi(x_1)^\top - \\ \vdots \\ -\phi(x_N)^\top - \end{pmatrix} \in \mathbb{R}^{N \times D}$.
- Weight vector: $\boldsymbol{w} \in \mathbb{R}^D$.
- Predictions: $\hat{\boldsymbol{y}} = \Phi \boldsymbol{w} \in \mathbb{R}^N$.

Example — fitting a polynomial:

$$f(x) = \sum_{i=0}^{M} \beta_i x^i, \qquad \phi(x) = (1,\ x,\ x^2, \ldots,\ x^M)^\top \in \mathbb{R}^{M+1}.$$

For 8 data points with $M = 7$ (8 parameters), the polynomial *interpolates exactly* (zero training error) but oscillates wildly between points — classic overfitting.

Least Squares Derivation

OLS Derivation (Full Matrix Calculus)

Minimise the squared-error loss:

$$L(\boldsymbol{w}) = \sum_{i=1}^{N} (y_i - \boldsymbol{w}^\top \phi(x_i))^2 = \|\boldsymbol{y} - \Phi \boldsymbol{w}\|_2^2.$$

Expand:

$$L(\boldsymbol{w}) = (\boldsymbol{y} - \Phi \boldsymbol{w})^\top (\boldsymbol{y} - \Phi \boldsymbol{w}) = \boldsymbol{w}^\top \Phi^\top \Phi \boldsymbol{w} - 2\boldsymbol{w}^\top \Phi^\top \boldsymbol{y} + \boldsymbol{y}^\top \boldsymbol{y}.$$

Differentiate w.r.t. $\boldsymbol{w}$ (using $\frac{\partial}{\partial \boldsymbol{w}} \boldsymbol{w}^\top A \boldsymbol{w} = 2A\boldsymbol{w}$ and $\frac{\partial}{\partial \boldsymbol{w}} \boldsymbol{w}^\top \boldsymbol{b} = \boldsymbol{b}$):

$$\frac{\partial L}{\partial \boldsymbol{w}} = 2\Phi^\top \Phi \boldsymbol{w} - 2\Phi^\top \boldsymbol{y}.$$

Set to zero and solve:

$$\boxed{\boldsymbol{w}_{\text{OLS}} = (\Phi^\top \Phi)^{-1} \Phi^\top \boldsymbol{y} = \Big(\sum_{i=1}^{N} \phi(x_i)\phi(x_i)^\top\Big)^{-1} \sum_{i=1}^{N} y_i\, \phi(x_i).}$$

△ OLS fails when $D > N$

$\Phi^\top\Phi \in \mathbb{R}^{D\times D}$ has rank at most $\min(N, D)$. When $D > N$, $\Phi^\top\Phi$ is **singular** (rank-deficient) and cannot be inverted. The system is underdetermined: infinitely many $\boldsymbol{w}$ achieve zero training loss. This motivates regularisation.

Regularisation

Empirical Risk Minimisation

The General Framework

$$\min_{\boldsymbol{w}} \; L(\boldsymbol{w}) = \sum_{i=1}^{N} \ell\big(\boldsymbol{w}^\top\phi(x_i),\; y_i\big) + \lambda\, r(\boldsymbol{w}).$$

$\ell(\cdot, y)$ is a per-instance loss; $r(\boldsymbol{w})$ is the regulariser; $\lambda \geq 0$ controls the trade-off (set via validation).

Ridge Regression (L_2 Regularisation)

Ridge Regression — Full Derivation

Objective:

$$L(\boldsymbol{w}) = \sum_{i=1}^{N}(y_i - \boldsymbol{w}^\top\phi(x_i))^2 + \lambda\,\boldsymbol{w}^\top\boldsymbol{w} = \|\boldsymbol{y} - \Phi\boldsymbol{w}\|^2 + \lambda\|\boldsymbol{w}\|^2.$$

Expand:

$$L = \boldsymbol{w}^\top(\Phi^\top\Phi + \lambda I)\boldsymbol{w} - 2\boldsymbol{w}^\top\Phi^\top\boldsymbol{y} + \boldsymbol{y}^\top\boldsymbol{y}.$$

Differentiate:

$$\frac{\partial L}{\partial \boldsymbol{w}} = 2(\Phi^\top\Phi + \lambda I)\boldsymbol{w} - 2\Phi^\top\boldsymbol{y} = \boldsymbol{0}.$$

Solve:

$$\boxed{\boldsymbol{w}_{\text{Ridge}} = (\Phi^\top\Phi + \lambda I)^{-1}\Phi^\top\boldsymbol{y}.}$$

Key properties:

- $(\Phi^\top\Phi + \lambda I)$ is **always invertible** for $\lambda > 0$, even when $D > N$.
- Shrinks all weights toward zero but never exactly to zero.
- Penalises large weights heavily (quadratic penalty).
- Small weights receive only a mild penalty ⇒ they persist.

LASSO (L_1 Regularisation)

LASSO

$$L(\boldsymbol{w}) = \sum_{i=1}^{N}(y_i - \boldsymbol{w}^\top\phi(x_i))^2 + \lambda\,\|\boldsymbol{w}\|_1 = \|\boldsymbol{y} - \Phi\boldsymbol{w}\|^2 + \lambda\sum_{d=1}^{D}|w_d|.$$

Key properties:

- Non-differentiable at $w_d = 0$ ⇒ requires specialised optimisers (coordinate descent, proximal gradient).
- Produces **sparse** solutions: some w_d go exactly to zero ⇒ automatic feature selection.

- Penalises all weights equally in absolute magnitude.

L_1 versus L_2: Detailed Comparison

L_1 vs. L_2 Regularisation

Consider the regulariser $r(\boldsymbol{w}) = \sum_d |w_d|^q$ parameterised by q:

- $q = 0$: counting regulariser (number of nonzero entries) — NP-hard to optimise.
- $q < 1$: non-convex objective.
- $q = 1$ **(LASSO)**: convex, sparse solutions, diamond-shaped constraint region.
- $q = 2$ **(Ridge)**: convex, closed-form, circular constraint region.

Geometric intuition: the LASSO constraint region (diamond) has corners on the axes. The loss contours are elliptical. The intersection of an ellipse with a diamond typically occurs at a corner (where one coordinate is zero), producing sparsity. The ridge constraint region (circle) has no corners, so intersections generically have all coordinates nonzero.

Behaviour with Redundant (Identical) Features

Suppose features d_1 and d_2 are identical: $\phi_{d_1}(x) = \phi_{d_2}(x)$ for all x.
OLS: $\Phi^\top \Phi$ is rank-deficient (singular). The loss depends only on $w_1 + w_2$, so infinitely many solutions exist.
Ridge: among all $\boldsymbol{w}$ with the same $w_1 + w_2$, the L_2 penalty $w_1^2 + w_2^2$ is minimised when $w_1 = w_2$. Ridge **splits weight equally**.
LASSO: the L_1 penalty $|w_1| + |w_2|$ is minimised when all weight is on one feature and the other is zero. LASSO **selects one arbitrarily** and zeros the other.

△ LASSO $w_d = 0$ does NOT mean "feature d is unimportant"

For correlated features, LASSO arbitrarily selects one and zeros the others. Both may be independently good predictors. Be careful interpreting zero coefficients as evidence of irrelevance.

Summary: The Mix-and-Match Framework

Model	Loss ℓ	Regulariser $r(\boldsymbol{w})$
OLS	Squared error	None
Ridge Regression	Squared error	$\|\boldsymbol{w}\|_2^2$ (L_2)
LASSO	Squared error	$\|\boldsymbol{w}\|_1$ (L_1)
Elastic Net	Squared error	$\alpha\|\boldsymbol{w}\|_1 + (1-\alpha)\|\boldsymbol{w}\|_2^2$
Logistic Regression	Log-loss	$\|\boldsymbol{w}\|_2^2$ (typically)
SVM	Hinge	$\|\boldsymbol{w}\|_2^2$

Linear Classification

Binary Classification Losses

For labels $y_i \in \{+1, -1\}$ and linear score $s_i = \boldsymbol{w}^\top \phi(x_i)$:

Logistic and Hinge Losses

Logistic loss (used in logistic regression):

$$\ell(s_i, y_i) = \log\big(1 + \exp(-s_i \cdot y_i)\big).$$

Smooth and differentiable; output can be interpreted as probability via $\sigma(s) = 1/(1 + e^{-s})$.
Hinge loss (used in SVM):

$$\ell(s_i, y_i) = \max\big(1 - s_i \cdot y_i,\ 0\big).$$

Non-differentiable at $s_i y_i = 1$; zero loss when $s_i y_i \geq 1$ (correctly classified with margin ≥ 1). Both are convex upper bounds on the 0–1 loss $\mathbb{I}[s_i y_i < 0]$.

Support Vector Machine

SVM: Maximum Margin Classifier

Objective:

$$\min_{\boldsymbol{w},b} \sum_{i=1}^{N} \max\big(1 - y_i(\boldsymbol{w}^\top \phi(x_i) + b),\ 0\big) + \lambda \|\boldsymbol{w}\|_2^2.$$

Intuition: for linearly separable data, many hyperplanes can separate the classes. The SVM chooses the one that **maximises the margin** γ — the distance from the nearest data point to the hyperplane.
Support vectors: the data points that lie exactly on the margin ($y_i(\boldsymbol{w}^\top \phi(x_i)+b) = 1$). Only these affect the solution; all other points can be removed without changing the hyperplane.
Equivalent primal (slack) formulation:

$$\min_{\boldsymbol{w},b}\ \boldsymbol{w}^\top \boldsymbol{w} + C \sum_{i=1}^{N} \xi_i \quad \text{s.t.} \quad y_i(\boldsymbol{w}^\top \phi(x_i) + b) \geq 1 - \xi_i,\ \xi_i \geq 0.$$

$C > 0$ controls regularisation; ξ_i are slack variables. Identical to the hinge-loss formulation.

Multi-Class Classification (Softmax)

Softmax Classification

For K classes, use weight matrix $W \in \mathbb{R}^{K \times D}$, logits $\boldsymbol{z}_i = W^\top \phi(x_i) \in \mathbb{R}^K$, and softmax:

$$\hat{y}_{i,k} = \frac{\exp(z_{i,k})}{\sum_{j=1}^{K} \exp(z_{i,j})}.$$

Loss (cross-entropy with one-hot $\boldsymbol{y}_i$):

$$\ell(\hat{\boldsymbol{y}}_i, \boldsymbol{y}_i) = -\sum_{j=1}^{K} y_{i,j} \log \hat{y}_{i,j}.$$

Feature Engineering

Exam Alert: The importance of features

"Coming up with features is difficult, time-consuming, requires expert knowledge. 'Applied machine learning' is basically feature engineering." — Andrew Ng
How well a linear learner works depends on ϕ. If ϕ is inadequate:

- **Kernel methods**: create large (potentially infinite) implicit features.
- **Deep learning**: learn appropriate ϕ automatically.

Connections:

MAP & Bayesian interpretation Ridge regression is equivalent to MAP estimation with a Gaussian prior $p(\boldsymbol{w}) = \mathcal{N}(\mathbf{0}, \alpha^{-1}I)$, with $\lambda = \alpha/\beta$ (prior precision / noise precision). LASSO corresponds to a Laplace prior. See BDL notes.

Part II
Kernel Methods

Motivation: Beyond Linear Features

When data is not linearly separable (e.g. a spiral), we need nonlinear features. The general solution: map $x \in \mathbb{R}^d$ to a higher-dimensional feature space $\phi(x) \in \mathbb{R}^D$, $D \gg d$, and apply a linear model there.

Problem: D can be enormous. For all-monomial features up to degree d on d-dimensional input, the feature vector includes $1, x_1, \ldots, x_d, x_1x_2, \ldots, x_1x_2\cdots x_d$, giving $D = 2^d$ features.

The **kernel trick** lets us work in this space *without ever computing* $\phi(x)$ explicitly.

The Kernel Trick

The Representer Theorem (by construction)

Claim: w lies in the span of training features

For the squared-error loss, the optimal weights can be expressed as:

$$\boldsymbol{w} = \sum_{i=1}^{N} \alpha_i \, \phi(x_i) = \Phi^\top \boldsymbol{\alpha}, \quad \text{for some } \boldsymbol{\alpha} \in \mathbb{R}^N.$$

Proof by Induction on Gradient Descent

Base case: initialise $\boldsymbol{w}_0 = \mathbf{0} = \sum \alpha_i^0 \phi(x_i)$ with $\alpha_i^0 = 0$.
Inductive step: the gradient of the squared-error loss is

$$\frac{\partial L}{\partial \boldsymbol{w}} = \sum_{i=1}^{N} \underbrace{2(\boldsymbol{w}^\top \phi(x_i) - y_i)}_{\equiv \gamma_i} \phi(x_i) = \sum_{i=1}^{N} \gamma_i \, \phi(x_i).$$

If $\boldsymbol{w}_t = \sum \alpha_i^t \phi(x_i)$, then:

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \epsilon \sum_i \gamma_i^t \, \phi(x_i) = \sum_i \big(\alpha_i^t - \epsilon\gamma_i^t\big)\phi(x_i) = \sum_i \alpha_i^{t+1} \phi(x_i).$$

So the form is preserved at every step, with $\alpha_i^{t+1} = \alpha_i^t - \epsilon\gamma_i^t$, i.e. $\alpha_i^T = -\epsilon \sum_{\tau=0}^{T-1} \gamma_i^\tau$. □

Why This Matters: Inner Products Only

Since $\boldsymbol{w} = \sum_i \alpha_i \phi(x_i)$, predictions become:

$$\boldsymbol{w}^\top \phi(x_j) = \sum_{i=1}^{N} \alpha_i \, \phi(x_i)^\top \phi(x_j).$$

The loss also depends only on inner products:

$$L(\boldsymbol{\alpha}) = \sum_{i=1}^{N} \Big(\sum_{j=1}^{N} \alpha_j \, \phi(x_j)^\top \phi(x_i) - y_i \Big)^2.$$

We never need $\phi(x)$ explicitly — only $\phi(x_i)^\top \phi(x_j)$.

Kernel Functions and the Kernel Matrix

Definitions

Kernel function: $\kappa(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$.
Kernel matrix: $K \in \mathbb{R}^{N \times N}$ with $K_{ij} = \kappa(x_i, x_j)$. Equivalently, $K = \Phi\Phi^\top$.
We have traded:

	Primal	Dual (Kernelised)
Parameters	$\boldsymbol{w} \in \mathbb{R}^D$	$\boldsymbol{\alpha} \in \mathbb{R}^N$
Data representation	$\Phi \in \mathbb{R}^{N \times D}$	$K \in \mathbb{R}^{N \times N}$
Useful when	$D < N$	$D > N$ (or $D = \infty$)

Common Kernels

Standard Kernel Functions

- **Linear**: $\kappa(x, z) = x^\top z$. Equivalent to a standard linear model.
- **Polynomial**: $\kappa(x, z) = (1 + x^\top z)^d$. All monomials up to degree d; inner product computable in $O(d_{\text{input}})$ instead of $O\binom{d_{\text{input}}+d}{d}$.
- **RBF (Gaussian / Squared Exponential)**: $\kappa(x, z) = \exp\left(-\frac{\|x-z\|^2}{2\sigma^2}\right)$. Corresponds to an **infinitely large** feature space. Most popular general-purpose kernel.

Kernel Validity

When is a kernel valid?

A function κ is a valid kernel iff the kernel matrix K is **positive semi-definite (PSD)** for *any* set of data points. Equivalent conditions:

1. $\exists\, B$ such that $K = BB^\top$.
2. $\forall\, \boldsymbol{q} \in \mathbb{R}^N$: $\boldsymbol{q}^\top K \boldsymbol{q} \geq 0$.
3. All eigenvalues of K are ≥ 0.

Proof that $K = \Phi\Phi^\top$ is PSD: $\boldsymbol{q}^\top K \boldsymbol{q} = \boldsymbol{q}^\top \Phi\Phi^\top \boldsymbol{q} = \|\Phi^\top \boldsymbol{q}\|^2 \geq 0$. □

△ Not every symmetric matrix is a valid kernel

A real symmetric matrix can have negative eigenvalues ⇒ not PSD ⇒ not a valid kernel.

Building Kernels from Other Kernels

Composition Rules

Given valid kernels κ_1, κ_2, the following are all valid:

$$\begin{aligned}
&\kappa(x,z) = c\,\kappa_1(x,z) && (c>0)\\
&\kappa(x,z) = \kappa_1(x,z) + \kappa_2(x,z)\\
&\kappa(x,z) = \kappa_1(x,z)\cdot\kappa_2(x,z)\\
&\kappa(x,z) = f(x)\,\kappa_1(x,z)\,f(z) && (f:\mathbb{R}^d\to\mathbb{R})\\
&\kappa(x,z) = g(\kappa_1(x,z)) && (g \text{ polynomial, positive coefficients})\\
&\kappa(x,z) = \exp(\kappa_1(x,z))\\
&\kappa(x,z) = x^\top A z && (A\succeq 0)\\
&\kappa(x,z) = \kappa_3(\phi(x),\phi(z)) && (\phi:\mathbb{R}^d\to\mathbb{R}^M,\ \kappa_3 \text{ valid on } \mathbb{R}^M)
\end{aligned}$$

Kernelised Algorithms

Recipe for Kernelising an Algorithm

1. **Prove**: the solution lies in the span of training points, $\boldsymbol{w} = \sum \alpha_i \phi(x_i) = \Phi^\top \boldsymbol{\alpha}$.
2. **Rewrite**: all inputs only appear in inner products $\phi(x_i)^\top \phi(x_j)$.
3. **Substitute**: replace $\phi(x_i)^\top \phi(x_j)$ with $\kappa(x_i, x_j)$.

Kernel (Ridge) Regression

Kernel Ridge Regression — Derivation

From the primal: $\Phi^\top\boldsymbol{\alpha} = \boldsymbol{w} = (\Phi^\top\Phi)^{-1}\Phi^\top\boldsymbol{y}$.
Left-multiply both sides by $\Phi\Phi^\top\Phi$:

$$(\Phi\Phi^\top)(\Phi\Phi^\top)\boldsymbol{\alpha} = \Phi\underbrace{(\Phi^\top\Phi)(\Phi^\top\Phi)^{-1}}_{I}\Phi^\top\boldsymbol{y} \implies K^2\boldsymbol{\alpha} = K\boldsymbol{y} \implies \boldsymbol{\alpha} = K^{-1}\boldsymbol{y}.$$

With L_2 regularisation:

$$\boxed{\boldsymbol{\alpha} = (K+\lambda I)^{-1}\boldsymbol{y}.}$$

Cost: $O(N^3)$ for inverting $K \in \mathbb{R}^{N\times N}$.
Prediction at test point $x_\star$:

$$\hat{y} = \boldsymbol{w}^\top\phi(x_\star) = \boldsymbol{\alpha}^\top\Phi\phi(x_\star) = \boldsymbol{y}^\top(K+\lambda I)^{-1}\boldsymbol{k}_\star,$$

where $(\boldsymbol{k}_\star)_i = \kappa(x_i, x_\star)$.

Kernel SVM

Kernelised SVM

Substitute $\boldsymbol{w} = \Phi^\top\boldsymbol{\alpha}$ into the hinge-loss objective:

$$\min_{\boldsymbol{\alpha},b}\ \boldsymbol{\alpha}^\top K\boldsymbol{\alpha} + C\sum_{i=1}^{N}\max\big(1 - y_i(\boldsymbol{\alpha}^\top\boldsymbol{k}_i + b),\ 0\big),$$

where $\boldsymbol{k}_i$ is the i-th column of K.
Prediction: $h(x_\star) = \operatorname{sign}\big(\sum_{i=1}^{N}\alpha_i\,\kappa(x_i, x_\star) + b\big)$.

Most $\alpha_i = 0$; the nonzero ones are the **support vectors**. At test time, only these need to be stored.
Squared hinge loss: replacing hinge with $\max(\cdot, 0)^2$ enables second-order optimisers (Newton's method).
The **dual formulation** (from constrained optimisation):

$$\max_{\boldsymbol{\lambda}} \sum_i \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j K_{ij} \quad \text{s.t.} \quad \sum_i \lambda_i y_i = 0,\ 0 \le \lambda_i \le C.$$

Finite Approximations: Random Fourier Features

Random Fourier Features

For the RBF kernel $\kappa(x, z) = \exp(-\frac{1}{2}\|x - z\|^2)$, define:

$$\phi(x) = \cos(Ax + b) \in \mathbb{R}^D,$$

where $A \in \mathbb{R}^{D \times d}$ with $A_{ij} \sim \mathcal{N}(0, 1)$ and $b \in \mathbb{R}^D$ with $b_i \sim \text{Uniform}[0, 2\pi]$ are drawn **once and fixed**.
Then $\mathbb{E}[\phi(x)^\top \phi(z)] = \kappa(x, z)$. Variance decreases as D increases.
Trade-off: if $D \ll N$, random features are much cheaper than the full kernel ($O(ND)$ vs. $O(N^3)$). If $D > N$, you'd be better off with exact kernel methods.

Computational Cost Summary

	Memory	Computation
Primal (feature space)	$O(ND)$	$O(ND^2)$ or $O(ND)$ per GD step
Kernel (exact)	$O(N^2)$	$O(N^3)$ inversion
Random features	$O(ND)$	$O(ND)$ per GD step

Connections:

GP connection The predictive mean of a **Gaussian process** with kernel κ and noise variance σ^2 is identical to kernel ridge regression with $\lambda = \sigma^2$. The GP additionally provides a predictive *variance* (uncertainty).

Part III
Optimisation

Gradient Descent

Gradient Descent

Given a loss $L(\theta)$, we iteratively update:

$$\theta_{t+1} = \theta_t - \epsilon \nabla L(\theta_t).$$

Justification: first-order Taylor expansion $L(\theta+\delta) \approx L(\theta)+\delta^\top \nabla L(\theta)$. Choosing $\delta = -\epsilon \nabla L$:

$$L(\theta_{t+1}) \approx L(\theta_t) - \epsilon \|\nabla L(\theta_t)\|^2 \ \leq \ L(\theta_t).$$

Guaranteed decrease (for small enough ϵ) since $\|\nabla L\|^2 \geq 0$.

Learning rate: too large ⇒ oscillation/divergence. Too small ⇒ very slow convergence. Common to use a schedule (decay over time) or adaptive methods.

Local minima: for non-convex L, GD converges to a local minimum (dependent on initialisation). Note the distinction between converging to the optimal *function value* vs. the optimal *parameters* — we might be near the optimal loss but far from the optimal θ.

Stochastic & Mini-Batch Gradient Descent

SGD

When $L(\theta) = \frac{1}{N}\sum_{i=1}^{N} L_i(\theta)$, each full gradient costs $O(N)$. Instead, use a single example (or mini-batch of M):

$$\theta_{t+1} = \theta_t - \epsilon \nabla L_j(\theta_t).$$

Unbiased: $\mathbb{E}_j[\nabla L_j] = \nabla L$ (expected value equals the full gradient).
Mini-batch: $\nabla L \approx \frac{1}{M}\sum_{j=1}^{M} \nabla L_j$. Trade-off:

- Large M: less noise, faster convergence (in iterations), better GPU utilisation.
- Small M: faster parameter updates (wall-clock), more exploration.

△ SGD loss fluctuations are normal

Stochastic gradients are noisy. The loss may **increase** on individual iterations. This does NOT mean the algorithm is diverging. Monitor a moving average of the loss; do not stop training after a single increase.

Momentum

Momentum

$$\tilde{g}_{t+1} = \mu\, \tilde{g}_t - \epsilon \nabla L(\theta_t), \qquad \theta_{t+1} = \theta_t + \tilde{g}_{t+1}.$$

$\tilde{g}$ is an exponential moving average of past gradients. $\mu \in [0, 1)$ is the momentum coefficient.
Benefits:

1. **Reduces oscillation**: in narrow valleys, the perpendicular component of the gradient

oscillates and cancels, while the along-valley component accumulates.
2. **Accelerates convergence**: near the minimum, gradients shrink and vanilla GD slows; momentum carries past updates forward.
3. **Crosses saddle points**: at a saddle ($\nabla L \approx \mathbf{0}$), accumulated momentum pushes through.
4. **Averages noisy gradients**: particularly useful for SGD.

Adaptive Learning Rates

Adagrad

Per-parameter accumulated squared gradient:

$$G_{t,ii} = \sum_{\tau=1}^{t} g_{\tau,i}^2.$$

Update:

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \varepsilon}}\, g_{t,i}.$$

+ Adaptive per-parameter learning rate. Default $\eta = 1.0$ or 0.1 often works. Good for sparse features (rare features get larger updates).
− Accumulated G grows monotonically $\Rightarrow$ learning rate decays to zero and never recovers.

Fixes: **RMSprop** uses exponential moving average of g^2 instead of sum. **Adam** combines RMSprop with momentum.

Second-Order Methods

Newton's Method

Second-order Taylor: $L(\theta + \delta) \approx L(\theta) + \delta^\top \nabla L + \frac{1}{2}\delta^\top H \delta$.
Minimise w.r.t. δ: $\nabla_\delta = \nabla L + H\delta = \mathbf{0} \;\Rightarrow\; \delta = -H^{-1}\nabla L$.
Update: $\theta_{t+1} = \theta_t - \epsilon\, H^{-1}\nabla L(\theta_t)$.
Properties:

- Converges in one step for quadratic functions.
- Requires $H \succ 0$ for a descent direction; not guaranteed for non-convex L.
- $O(D^2)$ memory for H; $O(D^3)$ to solve $H^{-1}\nabla L$.
- Not useful for stochastic objectives (Hessian estimate is noisy).

Quasi-Newton & Gauss–Newton

Quasi-Newton (BFGS/L-BFGS): approximate H^{-1} iteratively using gradient differences:

$$\nabla L(\theta_{t+1}) \approx \nabla L(\theta_t) + B_t(\theta_{t+1} - \theta_t).$$

L-BFGS limits memory to a fixed number of past iterates. Practical for medium-scale convex problems.
Gauss–Newton: for sum-of-squares losses $L = \sum (y_n - f(\theta, x_n))^2$, the Hessian is approximated by:

$$B \approx 2\sum_{n=1}^{N} \nabla f_n \nabla f_n^\top,$$

dropping the second-derivative term (small near the optimum). Guaranteed PSD; cheaper than full Newton.

Optimisation Methods Summary

Method	Order	Cost/step	Stochastic?	Notes
GD	1st	$O(ND)$	No	Baseline
SGD	1st	$O(MD)$	Yes	Noisy; fast per-step
Momentum	1st	$O(MD)$	Yes	Reduces oscillation
Adagrad/Adam	1st	$O(MD)$	Yes	Adaptive ϵ per param
Newton	2nd	$O(D^3)$	No	Fast convergence; expensive
L-BFGS	$\sim$2nd	$O(D)$	No	Practical quasi-Newton
Gauss–Newton	$\sim$2nd	$O(ND^2)$	No	PSD approx; sum-of-squares

Part IV
Exam-Style Questions & Lecture Exercises

Exam-Style Questions with Model Answers

Exam Q&A:

Q1 — OLS Derivation (5 marks) **Q**: Starting from $L(\boldsymbol{w}) = \|\boldsymbol{y} - \Phi\boldsymbol{w}\|^2$, derive the closed-form OLS solution.
A: Expand: $L = \boldsymbol{w}^\top\Phi^\top\Phi\boldsymbol{w} - 2\boldsymbol{w}^\top\Phi^\top\boldsymbol{y} + \boldsymbol{y}^\top\boldsymbol{y}$. Differentiate: $\partial L/\partial\boldsymbol{w} = 2\Phi^\top\Phi\boldsymbol{w} - 2\Phi^\top\boldsymbol{y}$. Set to zero: $\Phi^\top\Phi\boldsymbol{w} = \Phi^\top\boldsymbol{y}$. Solve: $\boldsymbol{w} = (\Phi^\top\Phi)^{-1}\Phi^\top\boldsymbol{y}$ (assuming $\Phi^\top\Phi$ is invertible, i.e. $N \geq D$ and features are linearly independent).

Exam Q&A:

Q2 — Ridge vs LASSO (4 marks) **Q**: You have 100 features, many of which are correlated. Compare what happens under Ridge and LASSO regularisation.
A: **Ridge** (L_2): keeps all 100 features with nonzero coefficients; distributes weight among correlated features (if two features are identical, both get equal weight). The coefficient path is smooth as λ varies. **LASSO** (L_1): produces a sparse solution with many coefficients exactly zero; among correlated features, arbitrarily selects one and zeros the others. This gives automatic feature selection, but the choice among correlated features is unstable. For interpretation, be cautious: $w_d = 0$ does not mean feature d is unimportant.

Exam Q&A:

Q3 — Kernel Validity (3 marks) **Q**: Is the matrix $K = \begin{pmatrix} 2 & 3 \\ 3 & 1 \end{pmatrix}$ a valid kernel?

A: Check PSD. Eigenvalues: $\lambda_{1,2} = \frac{3 \pm \sqrt{9+8}}{2}$. Wait — use the determinant test: $\det(K) = 2 \cdot 1 - 3 \cdot 3 = 2 - 9 = -7 < 0$. Since the determinant is negative, one eigenvalue is negative, so K is **not PSD** and is **not a valid kernel**.

Exam Q&A:

Q4 — Kernel Ridge vs Primal Ridge (3 marks) **Q**: You have $N = 50$ data points in $D = 10{,}000$ dimensions. Should you use primal or kernelised ridge regression?
A: **Kernelised**. Primal requires inverting $(\Phi^\top\Phi + \lambda I) \in \mathbb{R}^{10000\times10000}$, cost $O(D^3) \approx 10^{12}$. Kernelised requires inverting $(K + \lambda I) \in \mathbb{R}^{50\times50}$, cost $O(N^3) = 125{,}000$. When $N \ll D$, the kernel formulation is dramatically cheaper.

Exam Q&A:

Q5 — Momentum (2 marks) **Q**: Give two reasons why momentum helps optimisation.
A: (1) It averages over recent noisy gradient estimates, reducing variance and smoothing the trajectory. (2) It accumulates velocity in consistent directions, allowing faster traversal of flat regions and passage through saddle points where the gradient is near zero.

Exam Q&A:

Q6 — SGD Fluctuation (2 marks) **Q**: After 10 iterations of decreasing loss, the loss increases on iteration 11. Should you stop training?
A: **No**. SGD uses stochastic mini-batch gradients whose variance causes fluctuations. A single increase does not indicate convergence or divergence. Continue training and monitor a moving average of the loss instead.

Lecture Exercises & Worked Solutions

Exercise 1: What Happens if $D > N$? (OLS)

Exam Alert: From Linear Models, Slide 17

Question: Our estimator is $\boldsymbol{w} = (\Phi^\top \Phi)^{-1} \Phi^\top \boldsymbol{y}$. Note $\Phi \in \mathbb{R}^{N \times D}$. What happens if $D > N$?

Solution: $\Phi^\top \Phi \in \mathbb{R}^{D \times D}$ has rank at most $\min(N, D) = N < D$. Therefore $\Phi^\top \Phi$ is **singular** (rank-deficient) and $(\Phi^\top \Phi)^{-1}$ does not exist.

Consequence: the linear system $\Phi^\top \Phi \boldsymbol{w} = \Phi^\top \boldsymbol{y}$ is underdetermined. There are infinitely many solutions $\boldsymbol{w}$ that all achieve the *same* (minimum) training loss. Among these, the pseudo-inverse $\boldsymbol{w} = \Phi^\dagger \boldsymbol{y}$ gives the minimum-norm solution, but it is not regularised and typically overfits.

Exercise 2: What Happens if $D > N$? (Ridge)

Exam Alert: From Linear Models, Slide 22

Question: For ridge regression with $\boldsymbol{w} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top \boldsymbol{y}$, what happens if $D > N$?

Solution: Adding λI ($\lambda > 0$) shifts all eigenvalues of $\Phi^\top \Phi$ by $+\lambda$. Even if $\Phi^\top \Phi$ has zero eigenvalues (rank-deficient), $\Phi^\top \Phi + \lambda I$ has *all* eigenvalues $\geq \lambda > 0$, so it is **strictly positive definite** and always invertible.

Ridge regression gives a unique, well-defined solution even when $D > N$. The regularisation controls the trade-off: larger λ shrinks coefficients more, reducing overfitting at the cost of increased bias.

Exercise 3: Redundant Features

Exam Alert: From Linear Models, Slides 23–26

Question: What happens if you have two identical features in your model? Compare OLS, Ridge, and LASSO.

Solution: Let features d_1, d_2 be identical: $\phi_{d_1}(x) = \phi_{d_2}(x)$ for all x.

OLS: the loss $\sum (y_i - w_1 x_{i,1} - w_2 x_{i,2})^2$ depends only on $w_1 + w_2$ (since $x_{i,1} = x_{i,2}$). $\Phi^\top \Phi$ is singular. Infinitely many solutions parameterised by $w_1 + w_2 = c^*$.

Ridge: for a fixed sum $w_1 + w_2 = c$, the L_2 penalty $w_1^2 + w_2^2$ is minimised when $w_1 = w_2 = c/2$ (by AM-QM inequality or Lagrange multipliers). Ridge **splits weight equally**.

LASSO: for a fixed sum $w_1 + w_2 = c$, the L_1 penalty $|w_1| + |w_2| = |w_1| + |c - w_1|$ is minimised at the boundary: either $w_1 = c, w_2 = 0$ or $w_1 = 0, w_2 = c$. LASSO **selects one and zeros the other** (arbitrarily). This motivates the idea that LASSO performs automatic feature selection — but with the caveat that the "selected" feature among correlated ones is arbitrary.

Exercise 4: Do You Care About the Coefficients?

Exam Alert: From Linear Models, Slide 32

Question: Can LASSO replace manual feature selection? Do you care about the coefficients?

Solution: **Partially yes, with caveats**.

If you only care about prediction: LASSO works well — it selects a sparse subset of features that collectively predict well, and the resulting model is fast and interpretable.

If you care about understanding which features matter: LASSO can be misleading. For highly correlated features, only one will be selected (the others get $w_d = 0$), but *all* of them may independently be good predictors. $w_d = 0$ should **not** be interpreted as "feature d is unimportant."

Alternatives: Elastic Net ($L_1 + L_2$) tends to select groups of correlated features together. Stability selection (running LASSO many times on bootstrapped data and checking which features are consistently selected) provides more reliable feature importance.

Exercise 5: Feature Space Dimensionality (Kernels Quiz)

Exam Alert: From Kernels, Slide 5

Quiz: For input $x \in \mathbb{R}^d$, the all-monomials feature map is

$$\phi(x) = (1,\ x_1, \ldots, x_d,\ x_1x_2, \ldots, x_{d-1}x_d,\ \ldots,\ x_1x_2\cdots x_d)^\top.$$

What is the dimensionality D of this feature vector?

Solution: Each monomial is a product of a *subset* of the d input features (including the empty subset, which gives the constant 1). There are 2^d subsets of $\{x_1, \ldots, x_d\}$, so:

$$\boxed{D = 2^d.}$$

This grows *exponentially* in d — making explicit computation infeasible for even moderate d. This is precisely why the kernel trick is valuable: the corresponding inner product can be computed in $O(d)$ time:

$$\phi(x)^\top \phi(z) = \prod_{k=1}^{d} (1 + x_k z_k).$$

Exercise 6: Show the RBF Kernel is Valid

Exam Alert: From Kernels, Slide 21

Exercise: Using the kernel composition rules, show that the RBF kernel $\kappa(x, z) = \exp\left(-\frac{\|x-z\|^2}{2\sigma^2}\right)$ is a valid kernel.

Solution: Expand the exponent:

$$-\frac{\|x - z\|^2}{2\sigma^2} = -\frac{x^\top x}{2\sigma^2} + \frac{x^\top z}{\sigma^2} - \frac{z^\top z}{2\sigma^2}.$$

So:

$$\kappa(x, z) = \underbrace{\exp\left(-\frac{\|x\|^2}{2\sigma^2}\right)}_{f(x)} \cdot \underbrace{\exp\left(\frac{x^\top z}{\sigma^2}\right)}_{\kappa_0(x,z)} \cdot \underbrace{\exp\left(-\frac{\|z\|^2}{2\sigma^2}\right)}_{f(z)}.$$

Now verify each piece:

1. $\kappa_{\text{lin}}(x, z) = x^\top z$ is a valid kernel (it equals $\Phi\Phi^\top$ with $\Phi = X$).
2. $c \cdot \kappa_{\text{lin}} = \frac{1}{\sigma^2} x^\top z$ is valid ($c = 1/\sigma^2 > 0$).
3. $\exp(\kappa)$ is valid whenever κ is valid (since $e^\kappa = \sum_{n=0}^{\infty} \kappa^n / n!$ and products/sums of valid kernels with positive coefficients are valid).
4. So $\kappa_0(x, z) = \exp(x^\top z / \sigma^2)$ is valid.
5. $f(x)\, \kappa_0(x, z)\, f(z)$ is valid for any function f (composition rule).

Therefore the RBF kernel is a valid kernel. □

Exercise 7: Derive Kernel Ridge Regression

Exam Alert: From Kernels, Slide 27

Exercise: Show that an L_2 regularised kernel regression yields $\boldsymbol{\alpha} = (K + \lambda I)^{-1} \boldsymbol{y}$.

Solution: The regularised primal solution is $\boldsymbol{w} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top \boldsymbol{y}$, and we know $\boldsymbol{w} = \Phi^\top \boldsymbol{\alpha}$.
Method 1 (direct substitution into the dual loss): The regularised loss in terms of $\boldsymbol{\alpha}$ is:

$$L(\boldsymbol{\alpha}) = \|\boldsymbol{y} - \Phi\Phi^\top \boldsymbol{\alpha}\|^2 + \lambda\, \boldsymbol{\alpha}^\top \Phi\Phi^\top \boldsymbol{\alpha} = \|\boldsymbol{y} - K\boldsymbol{\alpha}\|^2 + \lambda\, \boldsymbol{\alpha}^\top K \boldsymbol{\alpha}.$$

Differentiate:

$$\frac{\partial L}{\partial \boldsymbol{\alpha}} = -2K(\boldsymbol{y} - K\boldsymbol{\alpha}) + 2\lambda K\boldsymbol{\alpha} = -2K\boldsymbol{y} + 2K(K + \lambda I)\boldsymbol{\alpha} = \mathbf{0}.$$

Assuming K is invertible: $(K + \lambda I)\boldsymbol{\alpha} = \boldsymbol{y}$, so $\boldsymbol{\alpha} = (K + \lambda I)^{-1} \boldsymbol{y}$.
Method 2 (Woodbury identity): from the primal $\boldsymbol{w} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top \boldsymbol{y}$, multiply by Φ:

$$\Phi \boldsymbol{w} = \Phi (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top \boldsymbol{y}.$$

By the push-through identity: $\Phi(\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top = (\Phi\Phi^\top + \lambda I)^{-1} \Phi\Phi^\top = (K + \lambda I)^{-1} K$.
So $K\boldsymbol{\alpha} = (K + \lambda I)^{-1} K \boldsymbol{y}$, giving $\boldsymbol{\alpha} = (K + \lambda I)^{-1} \boldsymbol{y}$. □

End of Week 2 Revision Notes.